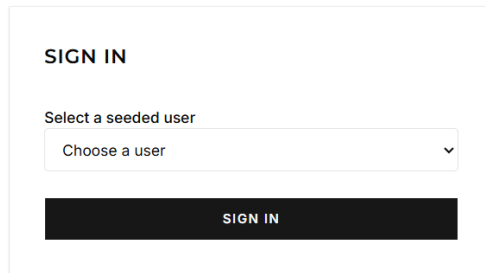


STORY 1

Log In As A Seeded User

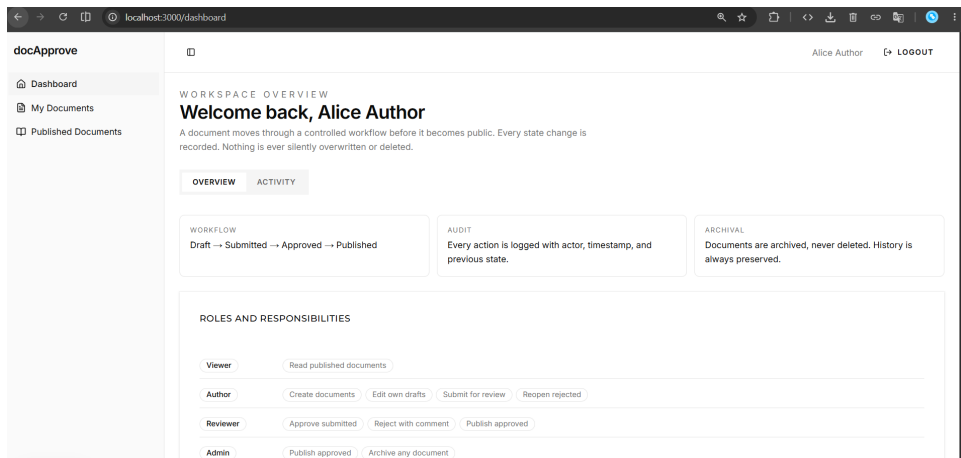
Proves that authentication is enforced server-side — not just hidden in the UI. The server rejects unauthenticated requests (401) and wrong-role requests (403) regardless of what the client sends.

Evidence 1 — Login page: seeded user dropdown



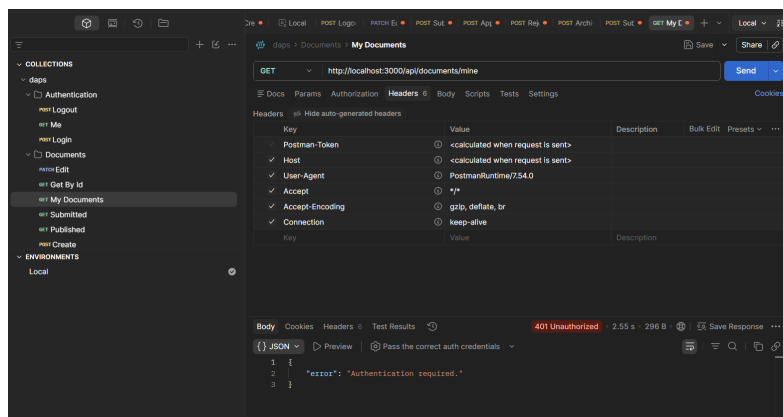
The login page presents all 5 seeded users in a dropdown. No password required — selecting a user creates a server-side session.

Evidence 2 — Logged-in header showing name and role



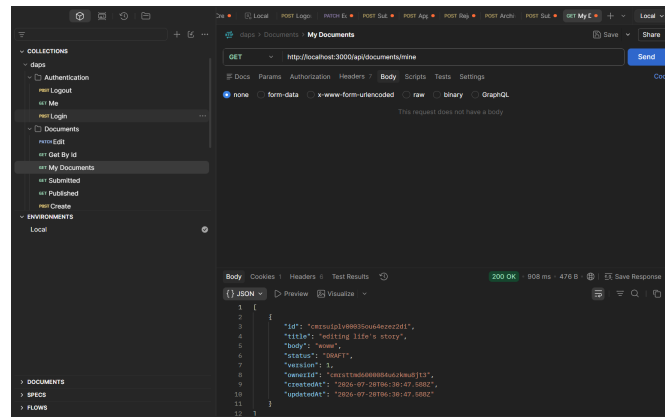
After logging in as Alice Author, the header shows 'Alice Author' and the sidebar shows only Author-scoped routes (My Documents, Published Documents).

Evidence 3 — GET /api/documents/mine without cookie → 401



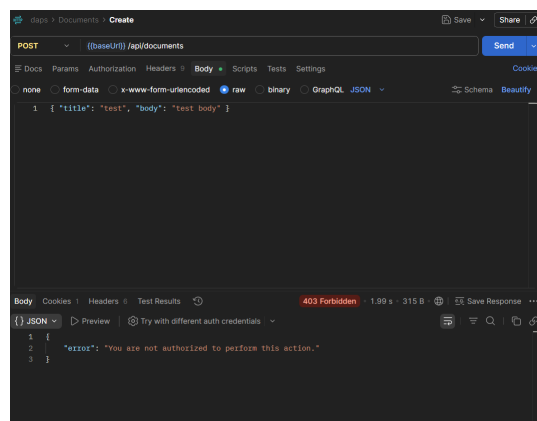
No Cookie header sent. Server returns 401 with { "error": "Authentication required." }.

Evidence 4 — Same route with valid session cookie → 200 OK



With a valid session cookie, the server resolves the user and returns the document list. Status 200 OK.

Evidence 5 — POST /api/documents as Viewer (wrong role) → 403



Viewer's session cookie sent. Server calls `assertAuthor()`, which throws `AuthorizationError` → 403 { "error": "You are not authorized to perform this action." }. The UI button never even existed for this user.

STORY 2

Create A Draft Document

Proves that document creation sets status=DRAFT and ownerId from the session, that empty title/body is rejected server-side, and that a CREATED audit event is written atomically with the document row.

Evidence 1 — Create Document form and resulting draft in the UI

The screenshot shows the 'docApprove' web application. On the left is a sidebar with navigation links: Dashboard, My Documents, and Published Documents. The main area is titled 'AUTHOR WORKSPACE' and 'My Documents'. It features a 'CREATE A DOCUMENT' form with a title field containing 'a blank canvas' and a body field containing 'sunshine and rainbows'. Below the form is a 'CREATE DOCUMENT' button. To the right of the form is a 'DRAFTS' section showing a list of documents. The first document is 'editing life's story' with status 'DRAFT', version '1', and a timestamp 'Updated Jul 20, 2026, 12:00 PM'. It has 'EDIT' and 'SUBMIT' buttons. Below it is a 'SUBMITTED' section with the text 'No documents are currently under review.'

Title 'a blank canvas' and body 'sunshine and rainbows' entered. After clicking CREATE DOCUMENT the draft appears immediately in the Drafts section.

Evidence 2 — Prisma Studio: Document row (status=DRAFT, ownerId set)

The screenshot shows the Prisma Studio interface for the 'doc-workflow' database. The 'Document' table is selected, showing two rows. Both rows have status 'DRAFT' and version '1'. The first row has title 'editing life's story' and body 'wow'. The second row has title 'a blank canvas' and body 'sunshine and rainbows'. Both rows have an 'ownerId' field populated with a user ID.

id	title	body	status	version	ownerId	createdAt	updatedAt	auditEvents
carut8p1v0003Sou4azet...	editing life's story	wow	DRAFT	1	cmr1ttd00000004uizkm...	2026-07-20 06:38:47.588	2026-07-20 06:38:47.588	AuditEvents
carut8p1v0003Sou4azet...	a blank canvas	sunshine and rainbows	DRAFT	1	cmr1ttd00000004uizkm...	2026-07-20 06:38:58.713	2026-07-20 06:38:58.713	AuditEvents

Both documents show status=DRAFT, version=1, and ownerId populated with Alice's user ID.

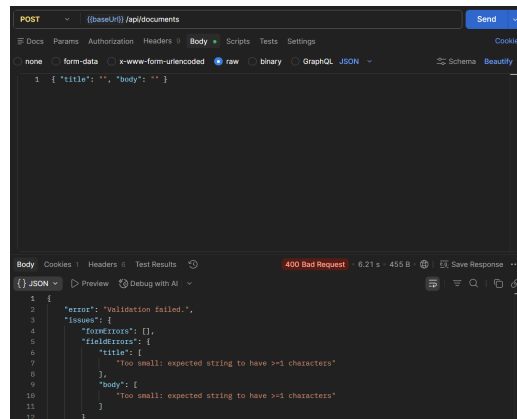
Evidence 3 — Prisma Studio: AuditEvent (action=CREATED, newStatus=DRAFT)

The screenshot shows the Prisma Studio interface for the 'doc-workflow' database. The 'AuditEvent' table is selected, showing one row. The row has action 'CREATED', previousStatus 'NULL', newStatus 'DRAFT', and a comment 'NULL'. It has a timestamp '2026-07-20 06:38:58.713' and a documentId 'carut8p1v0003Sou4azet...'. The documentId is linked to the 'Document' table.

id	action	previousStatus	newStatus	comment	createdAt	documentId
cmr1ttd00000004uizkm...	CREATED	NULL	DRAFT	NULL	2026-07-20 06:38:58.713	carut8p1v0003Sou4azet...

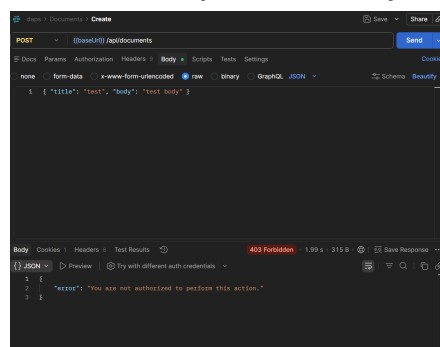
Inline AuditEvent for 'a blank canvas': action=CREATED, previousStatus=NULL, newStatus=DRAFT. Written atomically with the Document row inside db.\$transaction.

Evidence 4 — POST /api/documents with empty title+body → 400 Bad Request



Zod schema (`createDocumentSchema`) enforces `min(1)` on both fields. Returns 400 with `fieldErrors` on title and body.

Evidence 5 — POST /api/documents as Viewer → 403 (reused from Story 1 Evidence 5)



`assertAuthor()` in `documentService.create()` blocks the Viewer before any DB operation is attempted.

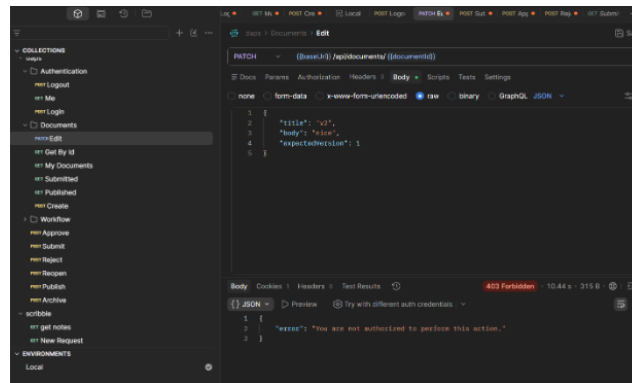
Edit A Draft Document

Evidence 1a — Edit form open on 'a blank canvas'

Alice clicks Edit. The inline form shows the current title and body with SAVE CHANGES / CANCEL controls.

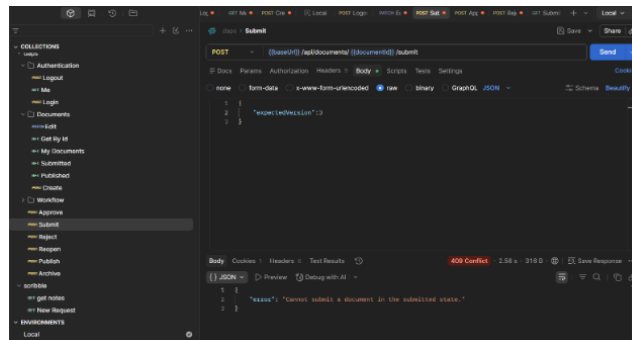
Document 'a blank canvas' now appears in the Submitted column at Version 3 (edited once → version 2, submitted → version 3), confirming version increments on every mutation.

Three events on 'a blank canvas': *CREATED* (null→DRAFT), *EDITED* (DRAFT→DRAFT), *SUBMITTED* (DRAFT→SUBMITTED). All share the same documentId.



`assertOwner()` check in `documentService.edit()` fires before any update. 403 returned even though Alice is a valid Author.

Evidence 4 — PATCH on already-SUBMITTED document → 409 Conflict



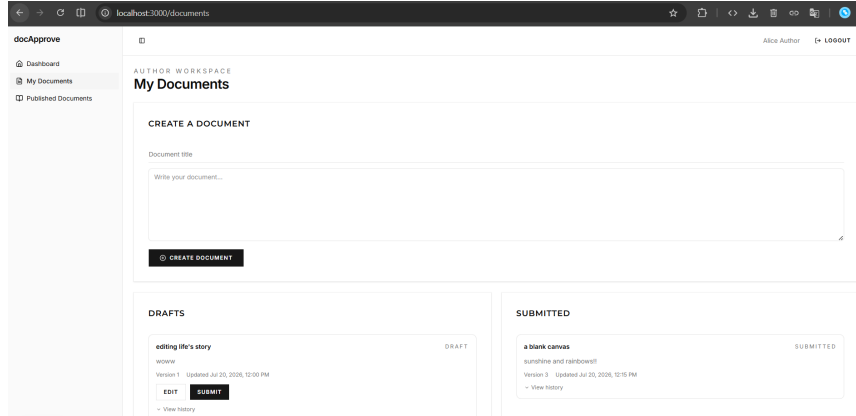
`assertDraft()` fires for the submit action (same pattern for edit). 409 { "error": "Cannot submit a document in the submitted state." }.

STORY 4

Submit A Document For Review

Proves the DRAFT→SUBMITTED transition is validated server-side, the document appears in the reviewer queue, and double-submission is blocked.

Evidence 1 — Draft visible in Author workspace; after Submit it moves to Submitted column



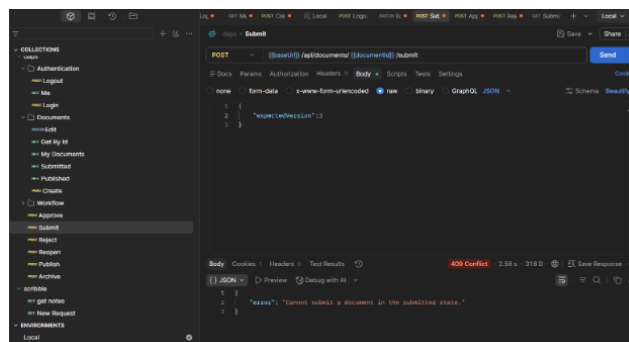
'a blank canvas' appears in the SUBMITTED column at Version 3. The Edit and Submit buttons are no longer shown — document is read-only to the author while submitted.

Evidence 2 — Prisma Studio: Document=SUBMITTED, AuditEvent action=SUBMITTED

AuditEvent						
id text	action AuditAction	previousStatus DocumentStatus	newStatus DocumentStatus	comment text	actorId text	documentId text
cmrsut9hm0075ou61slw78hl	CREATED	NULL	DRAFT	NULL	cmrstmd6000884u6zkmu8jt3	cmrsut8k600065ou6geum2a0
cmrsv0gq700095ou6shyo20mh	EDITED	DRAFT	DRAFT	NULL	cmrstmd6000884u6zkmu8jt3	cmrsut8k600065ou6geum2a0
cmrsv1lc9008aSou60b0dt1px	SUBMITTED	DRAFT	SUBMITTED	NULL	cmrstmd6000884u6zkmu8jt3	cmrsut8k600065ou6geum2a0

AuditEvent row: action=SUBMITTED, previousStatus=DRAFT, newStatus=SUBMITTED. Same actorId as the document owner, written in the same transaction.

Evidence 3 — POST /api/documents/:id/submit on already-SUBMITTED doc → 409



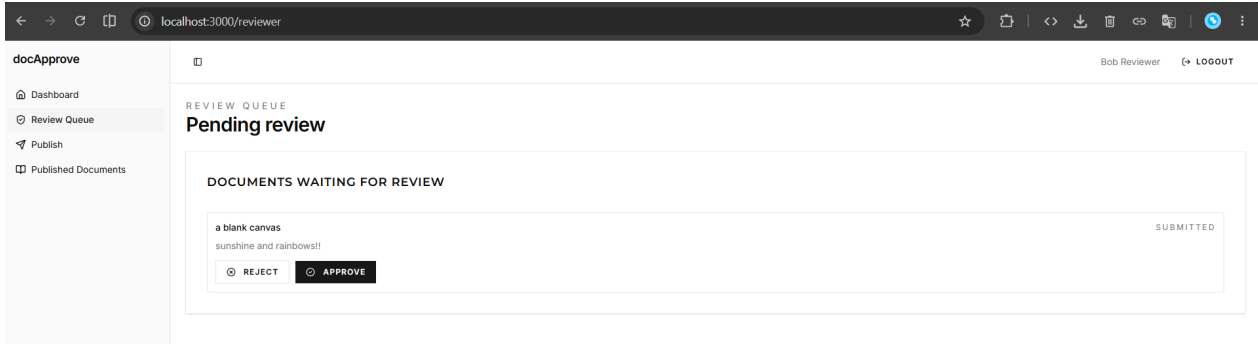
assertDraft() in documentService.submit() throws ConflictError. 409 { "error": "Cannot submit a document in the submitted state." }.

STORY 5

Review A Submitted Document (Approve / Reject)

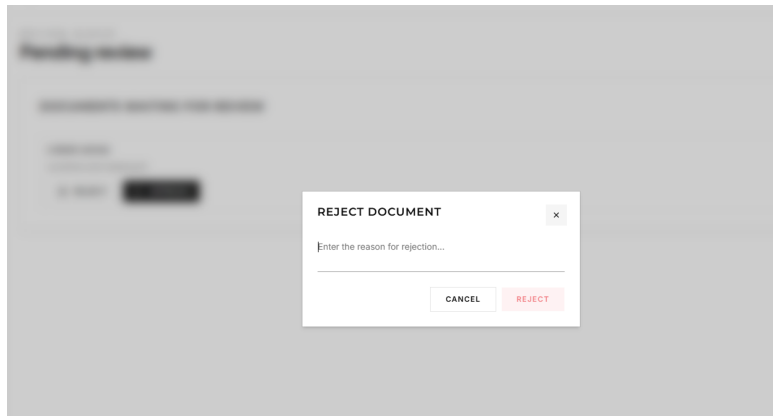
Proves that only Reviewers can approve/reject, the document owner cannot approve their own document, rejection requires a non-empty comment (enforced at both schema and DB constraint levels), and audit events are written for both actions.

Evidence 1 — Reviewer queue showing submitted document with Approve/Reject buttons



Bob Reviewer sees 'a blank canvas' as SUBMITTED with Reject and Approve buttons. Authors and Viewers do not see this page (requireReviewer() guard on the route).

Evidence 2 — Reject dialog requiring a comment before enabling Reject button



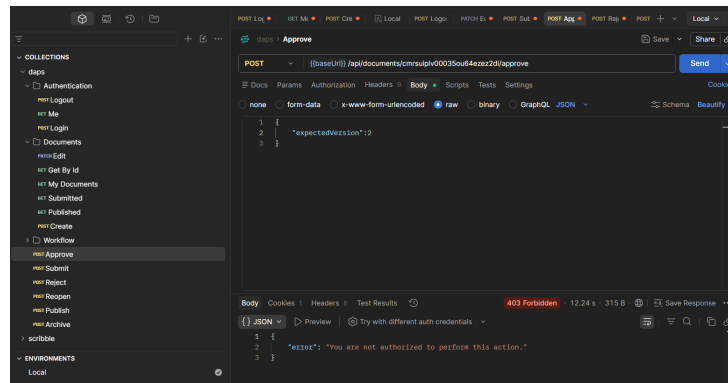
The Reject button is disabled (muted) until the textarea contains text. Even if bypassed in the UI, the server validates `comment.trim().length > 0` and the DB has a CHECK constraint on the AuditEvent table.

Evidence 3 — Prisma Studio: REJECTED event with comment populated

cmrsut8k600065ou6geum2...	a blank canvas	sunshine and rainbows!!	SUBMITTED	3	cmrsttd6000084u6zkmu8...	2026-07-20 06:38:58.713	2026-07-20 06:45:27.261	AuditEvents
AuditEvent								
id	action	previousStatus	newStatus	comment	actorId	documentId		
cmrsut9hm0075ou615lw7bh1	CREATED	NULL	DRAFT	NULL	cmrsttd6000084u6zkmu8j13	cmrsut8k600065ou6geum2a0		
cmrsv6q700095ou6shyo20mh	EDITED	DRAFT	DRAFT	NULL	cmrsttd6000084u6zkmu8j13	cmrsut8k600065ou6geum2a0		
cmrsv1lc9000a5ou6ob0dt1px	SUBMITTED	DRAFT	SUBMITTED	NULL	cmrsttd6000084u6zkmu8j13	cmrsut8k600065ou6geum2a0		
cmrsvcoop000c5ou6ie3zy7w2	REJECTED	SUBMITTED	REJECTED	no punctuation	cmrsttn19000284u6qto78s01	cmrsut8k600065ou6geum2a0		

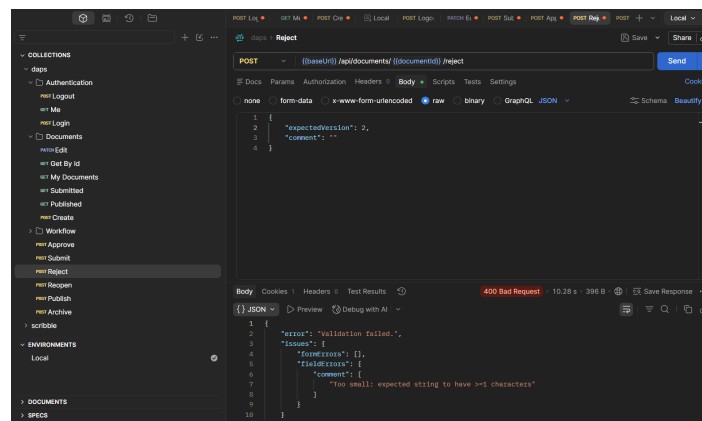
AuditEvent row: action=REJECTED, previousStatus=SUBMITTED, newStatus=REJECTED, comment='no punctuation'. actorId is Bob Reviewer's ID (different from Alice's ownerId).

Evidence 4 — POST /api/documents/:id/approve as document owner (Alice) → 403



Alice's session cookie. `documentService.approve()` checks `document.ownerId === actor.id` and throws `AuthorizationError`. Self-approval is blocked at the service layer.

Evidence 5 — POST /api/documents/:id/reject with empty comment → 400



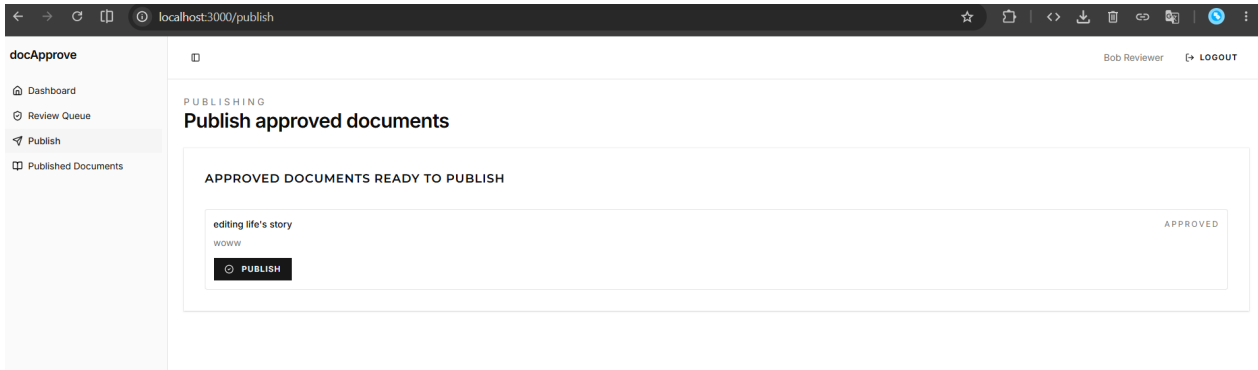
`rejectDocumentSchema` extends `actionDocumentSchema` with `comment: z.string().trim().min(1)`. Returns 400 with `fieldError: 'Too small: expected string to have >=1 characters'`.

STORY 6

Publish An Approved Document

Proves that only APPROVED documents can be published, only Reviewers or Admins can trigger the action, and viewers have no path to non-published documents.

Evidence 1 — Publish page (Bob Reviewer): approved document with Publish button



'editing life's story' shows as APPROVED on the Publish page. The Publish button triggers POST /api/documents/:id/publish.

Evidence 2 — Prisma Studio: Document=PUBLISHED, AuditEvent action=PUBLISHED

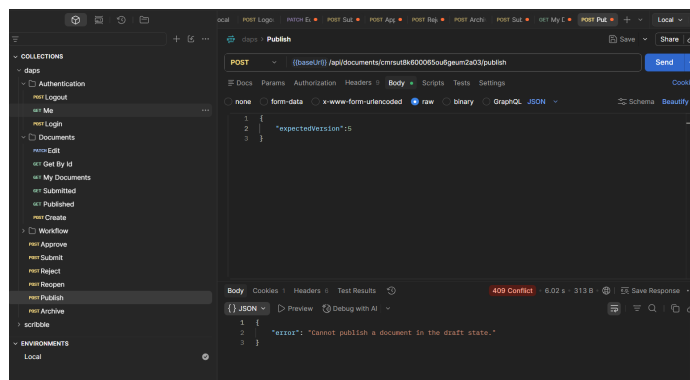
The screenshot shows the Prisma Studio interface. The document row has status=PUBLISHED, version=4. The AuditEvent chain is: CREATED → SUBMITTED → APPROVED → PUBLISHED (previousStatus=APPROVED, newStatus=PUBLISHED). actorId is Bob Reviewer.

id	title	body	status	version	ownerId	createdAt	updatedAt	AuditEvents
cmsuipiv00035ou64ez2d...	editing life's story	woww	PUBLISHED	4	cmsttmd600084u6zku8...	2026-07-20 06:30:47.588	2026-07-20 07:03:31.797	AuditEvents

id	action	previousStatus	newStatus	comment	actorId	documentId
cmsuipqc00045ou61i70j4ga	CREATED	NULL	DRAFT	NULL	cmsttmd600084u6zku8j3	cmsuipiv00035ou64ez2d
cmrsvf91000e5ou6ez3uinfh	SUBMITTED	DRAFT	SUBMITTED	NULL	cmsttmd600084u6zku8j3	cmsuipiv00035ou64ez2d
cmrsvnhk00015ou66t68e94e	APPROVED	SUBMITTED	APPROVED	NULL	cmsttnt9000284u6qto78s91	cmsuipiv00035ou64ez2d
cmrsvou1f000j5ou6l78noi1gb	PUBLISHED	APPROVED	PUBLISHED	NULL	cmsttnt9000284u6qto78s91	cmsuipiv00035ou64ez2d

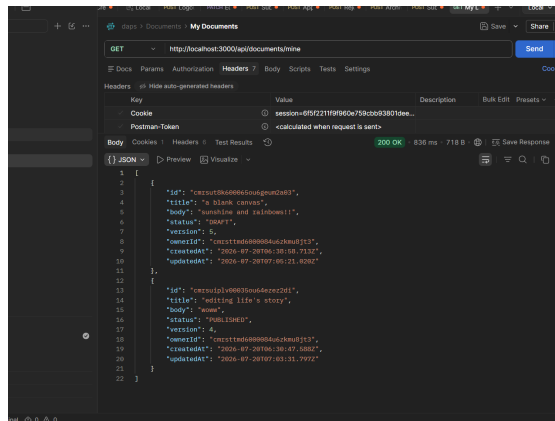
Document row: status=PUBLISHED, version=4. AuditEvent chain: CREATED → SUBMITTED → APPROVED → PUBLISHED (previousStatus=APPROVED, newStatus=PUBLISHED). actorId is Bob Reviewer.

Evidence 3 — POST /api/documents/:id/publish on DRAFT document → 409



assertApproved() in documentService.publish() fires. 409 { "error": "Cannot publish a document in the draft state." }.

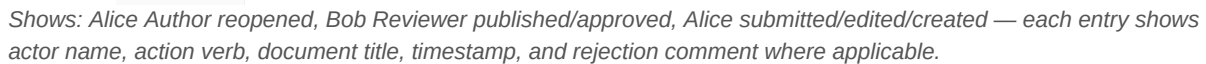
Evidence 4 — GET /api/documents/mine with author session: confirms per-user scoping



With Alice's session cookie, GET /api/documents/mine returns only Alice's own documents (200 OK, both DRAFT and PUBLISHED). A Viewer calling this same endpoint gets 403 (assertAuthor guard). Viewers are limited to GET /api/documents/published only.

View Audit History

Evidence 1 — Dashboard Activity tab: chronological feed across all documents



Five events shown: Created → Edited → Submitted for review → Rejected ('no punctuation') → Reopened. Each shows actor name, role in parentheses, and timestamp.

[illegible]

Five AuditEvent rows in ascending createdAt order: CREATED, EDITED, SUBMITTED, REJECTED (comment='no punctuation', different actorId), REOPENED. Matches the UI display exactly.

Transaction implementation

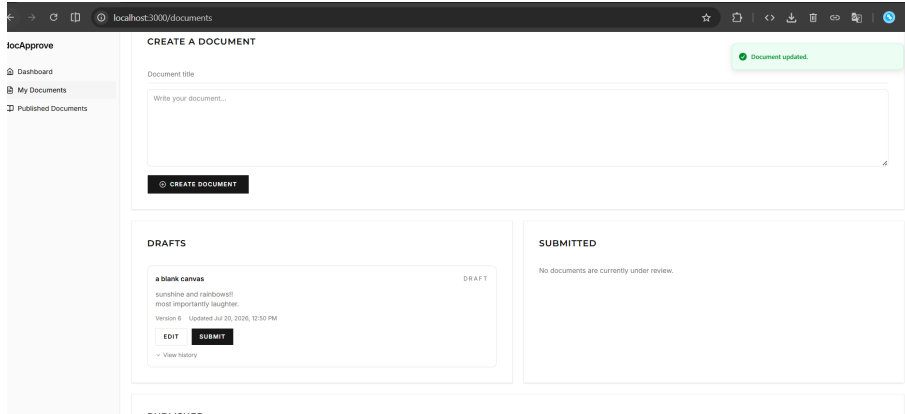
Every write function in **src/services/document.service.ts** (*create, edit, submit, approve, reject, reopen, publish, archive*) wraps both **documentRepository.update()** and **auditRepository.create()** inside a single **db.\$transaction(async (tx) => { ... })** block. Both repositories are instantiated with the transaction client **tx**, so if either write fails, the entire operation rolls back atomically. A document can never change state without a matching audit event, and an audit event can never exist for a state change that did not commit.

STORY 8

Handle Concurrent Updates

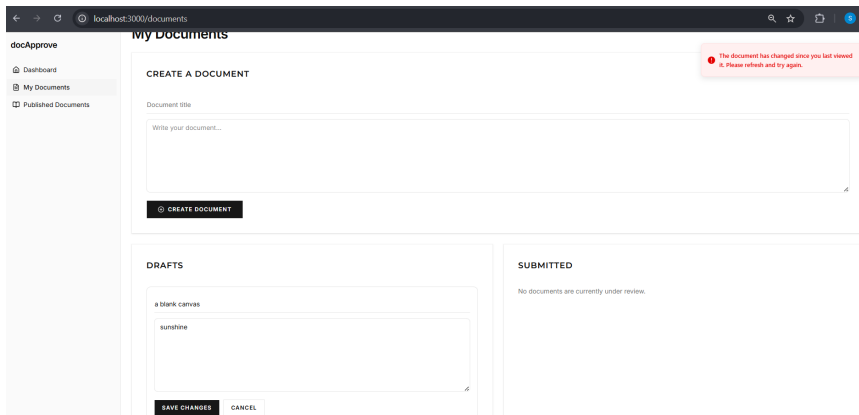
Proves that optimistic concurrency control prevents stale writes. Every mutation sends expectedVersion; the repository uses updateMany WHERE version=expectedVersion. If the version has moved on, count===0 and a 409 ConflictError is returned.

Evidence 1a — Tab 1: successful save, document advances to Version 6



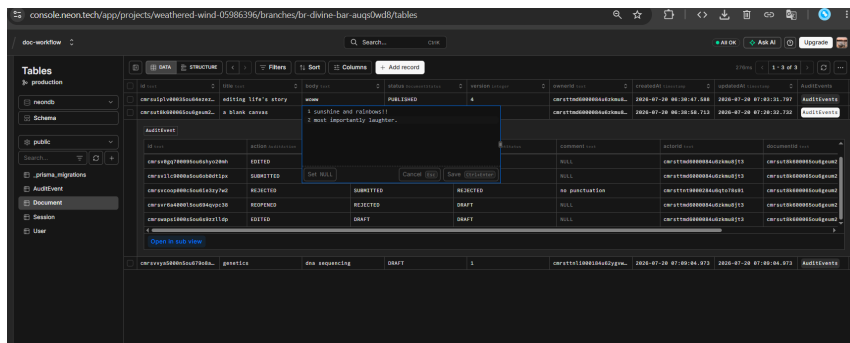
Tab 1 saves first. Green toast: 'Document updated.' Document now at Version 6 with body 'sunshine and rainbows!! most importantly laughter.'

Evidence 1b — Tab 2: stale page tries to save → conflict toast



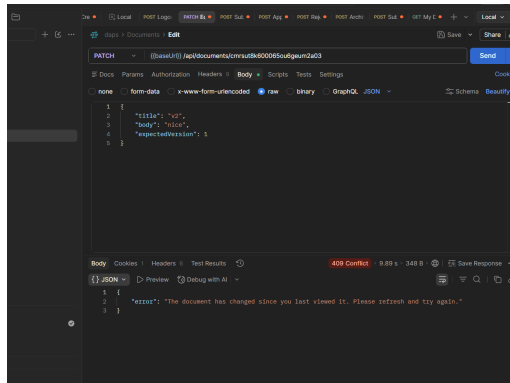
Tab 2 still holds version 5. Its SAVE CHANGES request is rejected. Red error toast: 'The document has changed since you last viewed it. Please refresh and try again.'

Evidence 2 — Prisma Studio: version column at 6, only one write landed



'a blank canvas' at version=6. The AuditEvent chain shows two EDITED rows (one per successful save), confirming exactly one write per version increment and no silent overwrite.

Evidence 3 — PATCH with stale expectedVersion:1 (doc is at version 6) → 409



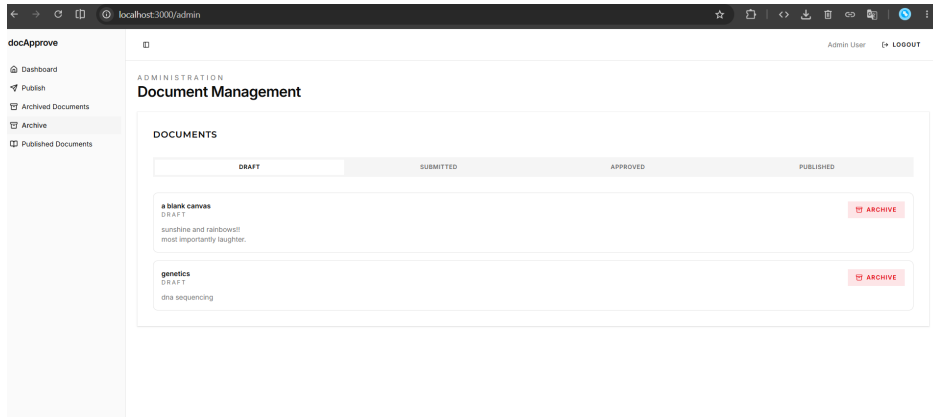
`documentRepository.update()` uses `updateMany WHERE { id, version: 1 }`. No row matches $\rightarrow$ `count===0` $\rightarrow$ `ConflictError` thrown. `409 { "error": "The document has changed since you last viewed it. Please refresh and try again." }`.

STORY 9

Archive A Document

Proves that Admins can archive, archiving is a soft delete (row preserved), an ARCHIVED audit event is written in the same transaction, and archived documents cannot be further mutated.

Evidence 1 — Admin panel: Document Management with Archive buttons



Admin User sees DRAFT, SUBMITTED, APPROVED, and PUBLISHED tabs. Each document has a red Archive button. This page is guarded by `requireAdmin()`.

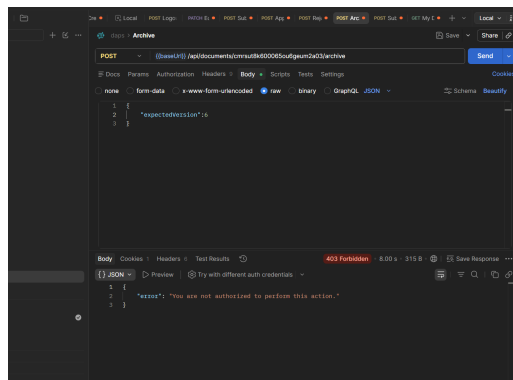
Evidence 2 — Prisma Studio: 'genetics' row status=ARCHIVED, row still exists

The screenshot shows the Prisma Studio interface with a table of documents. The 'genetics' row is highlighted, showing its status as 'ARCHIVED' and version as 2. The table columns include id, title, body, status, version, ownerId, createdAt, updatedAt, and AuditEvents.

id	title	body	status	version	ownerId	createdAt	updatedAt	AuditEvents
carsuip1v80835ou64ezex...	editing life's story	woww	PUBLISHED	4	carsttmd6080884u6zkmuS...	2026-07-20 06:38:47.588	2026-07-20 07:03:31.787	AuditEvents
carsut8k608085ou6geum2...	a blank canvas	sunshine and rainbows!	DRAFT	6	carsttmd6080884u6zkmuS...	2026-07-20 06:38:58.713	2026-07-20 07:28:32.732	AuditEvents
carsvvy5508n5ou679o8a...	genetics	dna sequencing	ARCHIVED	2	carsttn1808184u62ygvw...	2026-07-20 07:09:04.973	2026-07-20 07:38:28.507	AuditEvents

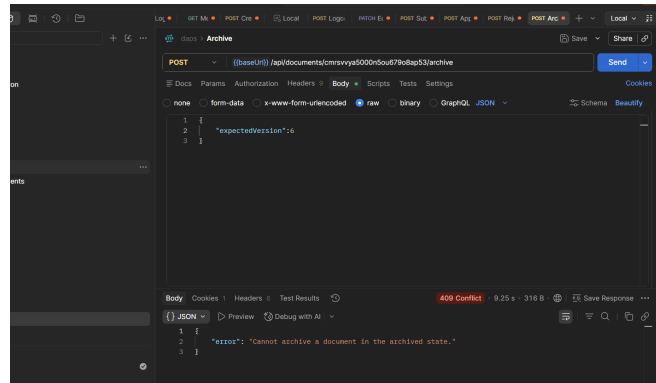
The 'genetics' row is present in the Document table with status=ARCHIVED, version=2. Not deleted — history and the document record are fully preserved.

Evidence 3 — POST /api/documents/:id/archive as non-Admin → 403



Alice's session cookie (AUTHOR role). `assertAdmin()` in `documentService.archive()` throws `AuthorizationError`. 403 { "error": "You are not authorized to perform this action." }.

Evidence 4 — POST /api/documents/:id/archive on already-ARCHIVED doc → 409



`assertArchivable()` allows only `DRAFT/SUBMITTED/APPROVED/PUBLISHED` → `ARCHIVED`. Attempting to re-archive returns `409 { "error": "Cannot archive a document in the archived state." }`.